CLASE 10 · PRESENTACIÓN DE LO REALIZADO

De los datos al modelo exportado

Predicción del precio de viviendas y comparación de valor por metro cuadrado

Autora: Griselda Blanco · Seminario de Actualización · IFTS N.º 18
Material de base: Cynthia M. Villagra · Revisión: 25/10/2026

Se ejecutó el recorrido de entrenamiento de una regresión lineal con tres entradas: superficie, presencia de balcón y presencia de ascensor. Se compararon implementaciones en scikit-learn, PyTorch y Keras, se exportaron los modelos y se verificó su recarga.

Como extensión, se aplicó el modelo exportado a ocho viviendas hipotéticas para comparar precio total, precio por m² y una elección con restricciones de presupuesto y superficie. Los valores provienen de inferencias ejecutadas, no de anuncios inmobiliarios.

Qué muestra esta presentación: decisiones, ejecución, resultados observados, problemas y límites. No reemplaza ni inventa las anticipaciones individuales que la consigna pide escribir antes de ejecutar cada celda.

01 · Qué modelo se usó y cómo se eligió

El nombre técnico es **regresión lineal múltiple**: aprendizaje supervisado para estimar una variable continua. La arquitectura tiene tres pesos y un bias. Se eligió a partir del ejemplo propuesto en clase, no mediante una búsqueda de modelos preentrenados en Hugging Face.

Herramienta	Función en el trabajo
LinearRegression (scikit-learn)	Referencia de mínimos cuadrados para ajustar la relación lineal.
SGDRegressor (scikit-learn)	Aprende la relación mediante correcciones iterativas con descenso de gradiente.
nn.Linear(3, 1) (PyTorch)	Una capa lineal: tres entradas y una salida. Se entrena con MSELoss y SGD.
Sequential + Dense(1) (Keras)	La misma función lineal con una API de alto nivel; fit ejecuta el ciclo.
NumPy, pandas y Matplotlib	Generación y organización de datos, cálculos y gráficos.
StandardScaler y joblib	Transformación de entradas; conservación del scaler y del pipeline.

Se consultaron las APIs oficiales para interpretar métodos, parámetros y formatos de guardado. Keras se ejecutó con backend PyTorch. Ningún modelo se seleccionó por una búsqueda de hiperparámetros: las configuraciones siguen el ejercicio docente.

02 · Datos, entrenamiento y evaluación

Se generaron **200 viviendas sintéticas** y se separaron 160 para entrenamiento y 40 para test. El precio se expresa en miles de dólares. El scaler se ajustó solo con train y luego se aplicó a test e inferencia.

Recorrido: entrada → predicción → pérdida MSE → gradiente → actualización de pesos. En PyTorch se utilizaron 200 épocas con SGD y tasa 0,1; Keras ejecutó 200 épocas con lote completo. SGDRegressor permitió observar otro recorrido iterativo. LinearRegression resolvió la referencia por mínimos cuadrados.

Implementación	RMSE (miles de US$)	R^2 en test
LinearRegression	4.423	0.9820
SGDRegressor	4.385	0.9823
PyTorch	4.423	0.9820
Keras	4.423	0.9820

RMSE mide la raíz del error cuadrático medio; aquí, alrededor de 4,4 miles de dólares. R^2 es cercano a 0,982 sobre este test sintético. Las diferencias mínimas entre implementaciones no demuestran que una sea superior. Estos resultados no validan una tasación en el mercado real.

03 · Qué se observó en las seis consignas

Consigna	Interpretación posterior de la ejecución
1. Estimar los pesos	En la escala original, el modelo PyTorch aprendido aporta aproximadamente US$1.503 por m², US$8.625 por balcón y US$12.565 por ascensor, manteniendo las otras entradas fijas. Son coeficientes del modelo, no efectos causales demostrados sobre viviendas reales.
2. Pesos al azar	En la primera casa del recorrido manual se obtuvo 14,84 frente a un precio real sintético de 93,8 (miles de US$). Tras una corrección, la predicción pasó a 42,73 y la pérdida bajó de 6.233,9 a 2.607,95.
3. SGD y mínimos cuadrados	Después de escalar, los métodos llegaron a ajustes y métricas similares. Es la misma familia de funciones aprendida con procedimientos diferentes.
4. Aumentar la tasa a 1,0	En ocho épocas la pérdida pasó de 22.152 a 22.652, con una baja inicial pequeña y crecimiento posterior. No se observó aprendizaje más rápido y estable.
5. El ciclo en Keras	Una llamada visible a fit ejecutó el ciclo que PyTorch presenta de forma explícita. Menos líneas visibles no significan menos operaciones internas.
6. Omitir el scaler	El regresor cargó sin excepción, pero predijo 2.422,7 miles de US$ para la casa de referencia, frente a unos 125,6 con el preprocesamiento correcto. Cargar un archivo no garantiza una predicción válida.

Esta tabla explica resultados ya observados. Las estimaciones personales previas de la estudiante son otra parte de la actividad y conservan su estado pendiente cuando no fueron escritas.

04 · Predicciones para ocho viviendas

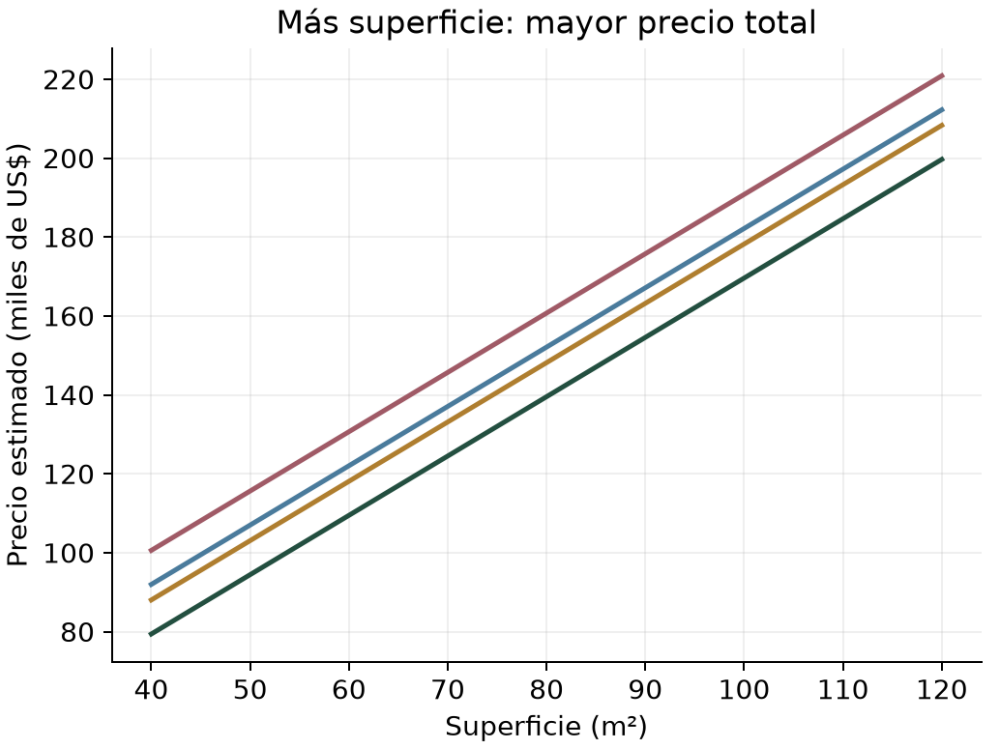
Se cargaron el modelo completo de PyTorch y su scaler, sin volver a entrenar. La salida en miles de dólares se multiplicó por 1.000; después se dividió por la superficie para obtener US$/m².

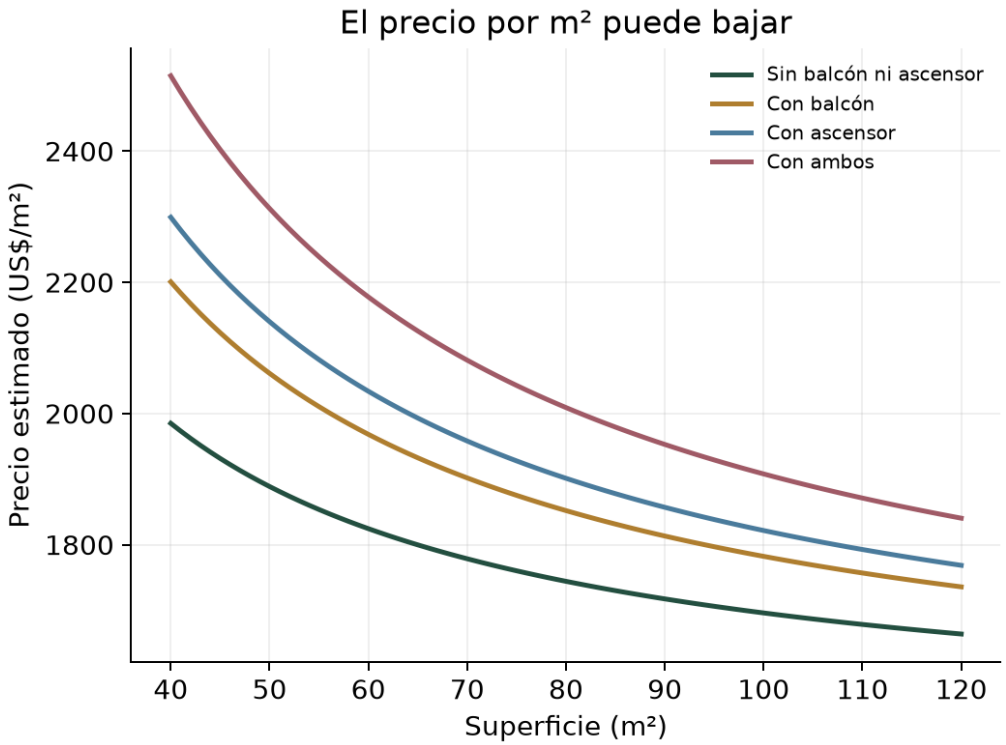
Casa	m²	Balcón	Ascensor	Precio estimado US$	US$/m²
A	40	No	No	79.407	1.985
B	65	No	No	116.992	1.800
C	90	No	No	154.577	1.718
D	120	No	No	199.679	1.664
E	65	Sí	No	125.618	1.933
F	65	No	Sí	129.557	1.993
G	65	Sí	Sí	138.183	2.126
H	120	Sí	Sí	220.870	1.841

Lectura: A es la de menor precio total. D tiene el menor precio estimado por m². H es la de mayor precio total. Los importes están redondeados; la comparación usa los valores sin redondear.

Para estudiar el efecto de los m², primero se comparan A, B, C y D, todas sin balcón ni ascensor. Para estudiar extras, se comparan B, E, F y G, todas de 65 m². Así no se confunde una diferencia de superficie con una de características.

05 · Más m² no implica mayor precio por m²





A igualdad de extras, la superficie aumenta el precio total. Sin embargo, el precio por m² disminuye en este modelo porque el costo fijo aprendido se reparte entre más metros.

Los gráficos muestran cuatro escenarios de extras dentro de 40 a 120 m². No incorporan ubicación, estado, antigüedad, expensas ni condiciones de una publicación real. El descenso por m² es consecuencia de la fórmula ajustada, no una regla universal del mercado.

06 · ¿Cuál sería la opción óptima?

“Óptima” requiere definir qué se quiere minimizar y qué restricciones se aceptan. Para este ejercicio se comparan tres criterios, usando solo las ocho viviendas hipotéticas.

Criterio explícito	Resultado	Cómo se interpreta
Menor desembolso total	A · 40 m² · US$79.407	Es la más barata en total, pero tiene menor superficie.
Menor precio por m², sin límite de presupuesto	D · 120 m² · US$1.664/m²	Ofrece el menor valor unitario del conjunto; su total es US$199.679.
Ejemplo: hasta US$160.000, al menos 60 m², minimizar US$/m²	C · 90 m² · US$154.577 · US$1.718/m²	Es la de menor valor unitario entre las que cumplen ambas restricciones.

Fórmula en dólares, aproximadamente:

precio = 1.503,40 × m² + 8.625,50 × balcón + 12.565,33 × ascensor + 19.271,38.

Al dividir por m², la contribución fija y la de los extras se reparten por superficie. Esto explica los gráficos. El presupuesto de US$160.000 y el mínimo de 60 m² son supuestos ilustrativos de este análisis, no preferencias reales informadas por la estudiante. “Óptima” aquí no equivale a mejor compra o inversión inmobiliaria.

07 · Exportación y problemas encontrados

El formato investigado fue **modelo completo de PyTorch**. torch.save conservó la instancia nn.Linear y sus tensores; el scaler se guardó aparte con joblib. Al recargar se compararon las predicciones antes y después: coincidieron sin ejecutar fit ni un nuevo entrenamiento.

Problema / decisión	Qué se hizo y qué se aprendió
Entradas con escalas diferentes	SGD sin escalar no arrojó una excepción en esta ejecución. No se confundió ausencia de error con convergencia; se escaló usando solo train para el recorrido estable.
Tasa de aprendizaje demasiado alta	Se conservaron las pérdidas de la prueba con tasa 1,0; la salida mostró crecimiento posterior al descenso inicial.
Modelo sin su transformación	Se documentó el resultado incorrecto de 2.422,7 frente a unos 125,6 miles de dólares. Se conservó el scaler para la carga correcta.
Compatibilidad de Keras	Se configuró KERAS_BACKEND=torch antes de importar Keras. Se usó la API de Keras sobre el motor PyTorch.
Seguridad del formato completo	weights_only=False se utilizó solo con el archivo generado en esta notebook. No es seguro cargar objetos pickle o joblib de desconocidos.

Un objeto completo no empaqueta necesariamente todo el código de una clase personalizada. Para continuar entrenamiento también haría falta el estado del optimizador y otra información del proceso. La inferencia de este ejemplo requiere el orden de entradas m2, balcon, ascensor.

08 · Evidencia, reproducción y conclusión

Archivos: notebook ejecutada; resultados_ejecucion.json con versiones y métricas; modelos_exportados/ con modelos y scalers; comparar_casas.py con la nueva inferencia; comparativa_casas.csv, resumen.json y el gráfico con los ocho escenarios.

Reproducir la comparación: desde la carpeta clase_10, con las dependencias instaladas, ejecutar `python comparar_casas.py`. Ese script carga los artefactos propios de confianza y hace inferencia; no entrena de nuevo. En un entorno local diferente se debe usar el ejecutable Python del entorno donde se instalaron las bibliotecas.

Conclusión: las tres bibliotecas permitieron ajustar una función lineal y exportarla. El preprocesamiento fue parte esencial de la predicción. El análisis adicional mostró que precio total y precio por m² pueden conducir a elecciones distintas; cualquier selección necesita un criterio explícito y límites de aplicación.

Los modelos se entrenaron con datos sintéticos. No hubo búsqueda de una vivienda real ni selección de un modelo preentrenado del Hub. Tampoco se midió rendimiento en datos inmobiliarios externos. El análisis es una extensión didáctica, no una validación comercial.

Fuentes y entrega

scikit-learn: LinearRegression · PyTorch: guardar y cargar modelos · Keras: Dense
Notebook docente proporcionada por la estudiante; salidas guardadas de su copia ejecutada. Consulta de documentación: 25/10/2026.
Repositorio: Seminario2026 / Proyecto_Integrador / clase_10.

Se utilizó asistencia de IA para ejecutar y verificar código, generar la comparación y redactar este documento. La estudiante debe revisar las conclusiones y completar sus respuestas individuales pendientes antes de entregar.